IPSSI

2e année Bachelor Informatique

Mathématiques appliquées à l'Intelligence Artificielle

Fiche d'exercices

Jour 4 — Réseaux de neurones

Auteur : Mohamed EL AFRIT

www.mohamedelafrit.com

Année académique : 2025–2026

© 2025 Mohamed EL AFRIT — IPSSI

Ce contenu est distribué sous licence

Creative Commons BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.fr

Rappels — Formules du Jour 4

Neurone : $a = \sigma(\mathbf{w}^T\mathbf{x} + b)$ **Sigmoïde :** $\sigma(z) = \frac{1}{1+e^{-z}}$ **ReLU :** $\mathrm{ReLU}(z) = \max(0, z)$

Forward pass : $\mathbf{h} = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$; $\hat{y} = \sigma(\mathbf{w}^{(2)T}\mathbf{h} + b^{(2)})$

Mise à jour : $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha\nabla L$ **Dérivée sigmoïde :** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Valeurs utiles : $\sigma(0) = 0{,}5$; $\sigma(1) \approx 0{,}73$; $\sigma(2) \approx 0{,}88$; $\sigma(-1) \approx 0{,}27$; $\sigma(-2) \approx 0{,}12$

Exercices crayon-papier — Niveau fondamental

Exercice 1 — ★☆☆ **Fondamental** — Sortie d'un neurone

Un neurone a les poids $\mathbf{w} = \begin{pmatrix} 0{,}3 \\ 0{,}5 \end{pmatrix}$, le biais $b = -2$ et utilise l'activation **sigmoïde**.

L'entrée est $\mathbf{x} = \begin{pmatrix} 5 \\ 4 \end{pmatrix}$ (expérience = 5, langages = 4).

a) Calculez la somme pondérée $z = \mathbf{w}^T\mathbf{x} + b$.

b) Calculez l'activation $a = \sigma(z) = \frac{1}{1+e^{-z}}$. *Conseil : calculez d'abord e^{-z} avec une calculatrice ou les valeurs utiles ci-dessus.*

c) Si le seuil de décision est 0,5, quelle classe le neurone prédit-il ?

d) On change l'entrée : $\mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$ (exp=1, lang=2). Recalculez z et a. Quelle classe ?

Exercice 2 — ★☆☆ **Fondamental** — Appliquer ReLU et sigmoïde

Calculez les valeurs suivantes en détaillant chaque étape :

a) $\sigma(0)$, $\sigma(2)$, $\sigma(-3)$

b) $\mathrm{ReLU}(2{,}5)$, $\mathrm{ReLU}(0)$, $\mathrm{ReLU}(-1{,}7)$

c) $\sigma(\mathrm{ReLU}(-1))$ — composez les deux fonctions.

d) $\sigma(\mathrm{ReLU}(3))$ — même chose.

e) Pourquoi $\sigma(z)$ ne vaut-il **jamais** exactement 0 ni exactement 1 ?

Exercice 3 — ★☆☆ **Fondamental** — Forward pass d'un réseau 2-2-1

On considère un réseau avec 2 entrées, 2 neurones cachés, 1 sortie. Tous les neurones utilisent l'activation sigmoïde.

Poids couche cachée :

$$\mathbf{W}^{(1)} = \begin{pmatrix} 0{,}4 & 0{,}6 \\ -0{,}3 & 0{,}5 \end{pmatrix} \quad \mathbf{b}^{(1)} = \begin{pmatrix} -1 \\ 0{,}5 \end{pmatrix}$$

Poids couche de sortie : $\mathbf{w}^{(2)} = \begin{pmatrix} 0{,}7 \\ 0{,}8 \end{pmatrix}$, $b^{(2)} = -0{,}5$.

Entrée : $\mathbf{x} = \begin{pmatrix} 3 \\ 5 \end{pmatrix}$.

a) Calculez les sommes pondérées de la couche cachée : $z_1 = \mathbf{W}^{(1)}_{\text{ligne } 1} \cdot \mathbf{x} + b_1^{(1)}$ et z_2.
b) Appliquez l'activation : $h_1 = \sigma(z_1)$ et $h_2 = \sigma(z_2)$.
c) Calculez la somme pondérée de la sortie : $z_{\text{out}} = \mathbf{w}^{(2)T}\mathbf{h} + b^{(2)}$.
d) Calculez la prédiction $\hat{y} = \sigma(z_{\text{out}})$.
e) Le réseau prédit-il classe 0 ou classe 1 ?

Exercice 4 — ★☆☆ **Fondamental** — Dimensions des matrices de poids

Un réseau a l'architecture suivante : **5 entrées → 8 neurones cachés → 3 neurones cachés → 1 sortie**.

a) Quelle est la dimension de $\mathbf{W}^{(1)}$ (poids de la 1re couche cachée) ? Et de $\mathbf{b}^{(1)}$?
b) Quelle est la dimension de $\mathbf{W}^{(2)}$ (poids de la 2e couche) ? Et de $\mathbf{b}^{(2)}$?
c) Quelle est la dimension de $\mathbf{w}^{(3)}$ (poids de la sortie) ?
d) Combien de paramètres (poids + biais) le réseau a-t-il **au total** ?
e) Si on augmente la couche cachée 1 de 8 à 16 neurones, combien de paramètres supplémentaires cela ajoute-t-il ?

Exercices crayon-papier — Niveau intermédiaire

Exercice 5 — ★★☆ **Intermédiaire** — Composer deux fonctions

On définit deux neurones :

— Neurone A : $f_A(x) = \text{ReLU}(2x - 1)$
— Neurone B : $g_B(x) = \sigma(3x - 2)$

a) Calculez $f_A(0)$, $f_A(1)$, $f_A(3)$.
b) Calculez la composition $g_B(f_A(x))$ pour $x = 0$, $x = 1$, $x = 3$. Détaillez chaque étape.
c) La composition $g_B \circ f_A$ est-elle une fonction linéaire ? Justifiez.
d) Que se passerait-il si les deux neurones n'avaient **pas** de fonction d'activation (i.e. $f_A(x) = 2x - 1$, $g_B(x) = 3x - 2$) ? Calculez la composition. Est-elle linéaire ?

Exercice 6 — ★★☆ **Intermédiaire** — Pourquoi les activations sont indispensables

On considère un réseau à 2 couches **sans** fonction d'activation :

$$\begin{aligned} \text{Couche 1 :} \quad & \mathbf{h} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \\ \text{Couche 2 :} \quad & \hat{y} = \mathbf{w}^{(2)T}\mathbf{h} + b^{(2)} \end{aligned}$$

a) Substituez l'expression de $\mathbf{h}$ dans celle de $\hat{y}$. Simplifiez l'expression.
b) Montrez que $\hat{y}$ peut s'écrire sous la forme $\hat{y} = \tilde{\mathbf{w}}^T\mathbf{x} + \tilde{b}$ pour un certain $\tilde{\mathbf{w}}$ et $\tilde{b}$.
c) Que représente cette forme ? Le réseau à 2 couches est-il plus puissant qu'un simple neurone ?

d) Conclusion : pourquoi les fonctions d'activation sont-elles **indispensables** dans un réseau multicouche ?

Exercice 7 — ★★☆ Intermédiaire — Analyser un cas de surapprentissage

Un étudiant a entraîné un réseau sur le dataset salaire et obtient les accuracies suivantes au fil des époques :

Époque	Accuracy entraînement	Accuracy validation
10	70%	67%
50	85%	83%
200	95%	87%
500	99%	82%
1000	100%	75%

a) À quelle époque le modèle commence-t-il à **surapprentir** ? Justifiez.
b) Quelle époque choisiriez-vous comme **meilleur modèle** ? Pourquoi ?
c) Proposez 3 solutions pour réduire le surapprentissage.
d) Pourquoi une accuracy de 100% sur l'entraînement est-elle **suspecte** plutôt que rassurante ?

Exercices Python guidés — Niveau intermédiaire

Exercice 8 — ★★☆ Intermédiaire — Sigmoïde et ReLU en Python [Python guidé]

» Squelette -- Fonctions d'activation

```python
import numpy as np
import matplotlib.pyplot as plt

# 1. Implementer la sigmoide
def sigmoid(z):
    return ______  # A COMPLETER

# 2. Implementer ReLU
def relu(z):
    return ______  # A COMPLETER

# 3. Tester sur des valeurs
valeurs = np.array([-3, -1, 0, 1, 2, 5])
print("z       :", valeurs)
print("sigmoid :", np.round(sigmoid(valeurs), 4))
print("relu    :", relu(valeurs))

# 4. Tracer les courbes
z = np.linspace(-6, 6, 200)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

```

```python
ax1.plot(z, ______, 'b-', lw=2)  # A COMPLETER
ax1.set_title("Sigmoide"); ax1.grid(True, alpha=0.3)
ax1.axhline(0.5, color='gray', ls=':')

ax2.plot(z, ______, 'r-', lw=2)  # A COMPLETER
ax2.set_title("ReLU"); ax2.grid(True, alpha=0.3)

plt.tight_layout(); plt.show()

# 5. Derivee de la sigmoide
def sigmoid_deriv(z):
    s = sigmoid(z)
    return ______  # A COMPLETER : s * (1 - s)

print("\nDerivee sigmoid(0) =", sigmoid_deriv(0))
print("Derivee sigmoid(2) =", round(sigmoid_deriv(2), 4))
```

Exercice 9 — ★★☆ **Intermédiaire** — Forward pass 2-2-1 en NumPy [Python guidé]

» Squelette -- Forward pass

```python
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Entree : un developpeur
x = np.array([5.0, 4.0])

# Poids couche cachee (2 neurones)
W1 = np.array([[0.3, 0.5],
               [-0.2, 0.4]])
b1 = np.array([-2.0, 0.5])

# Poids couche de sortie (1 neurone)
w2 = np.array([0.8, 0.6])
b2 = -0.5

# 1. Couche cachee
z1 = ______            # A COMPLETER : W1 @ x + b1
h  = ______            # A COMPLETER : sigmoid(z1)
print(f"z1 = {z1}")
print(f"h  = {np.round(h, 4)}")

# 2. Couche de sortie
z2 = ______            # A COMPLETER : w2 @ h + b2
y_pred = ______        # A COMPLETER : sigmoid(z2)
print(f"\nz2 = {z2:.4f}")
print(f"y_pred = {y_pred:.4f}")
```

```python
# 3. Decision
classe = ______        # A COMPLETER : 1 si y_pred > 0.5
print(f"\nClasse predite : {classe}")

# 4. Tester sur tous les developpeurs
print("\n--- Predictions sur 6 developpeurs ---")
devs = np.array([[0,2],[1,1],[5,4],[9,5],[12,7],[18,5]])
for d in devs:
    h_i = sigmoid(W1 @ d + b1)
    y_i = sigmoid(w2 @ h_i + b2)
    print(f"  x={d} -> y_pred={y_i:.3f} "
          f"-> classe {1 if y_i > 0.5 else 0}")
```

Exercice 10 — ★★☆ **Intermédiaire** — Réseau qui apprend XOR [Python guidé]

» Squelette -- XOR avec backpropagation

```python
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))
def sigmoid_d(a):
    return a * (1 - a)

# Donnees XOR
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([[0],[1],[1],[0]], dtype=float)

# Poids aleatoires (reseau 2-4-1)
np.random.seed(42)
W1 = np.random.randn(2, 4) * 0.5
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1) * 0.5
b2 = np.zeros((1, 1))

alpha = 1.0

for epoch in range(5000):
    # Forward
    z1 = ______          # A COMPLETER : X @ W1 + b1
    h  = ______          # A COMPLETER : sigmoid(z1)
    z2 = ______          # A COMPLETER : h @ W2 + b2
    out = ______         # A COMPLETER : sigmoid(z2)

    loss = np.mean((y - out) ** 2)

    # Backward
    d_out = (out - y) * sigmoid_d(out)
    dW2 = ______         # A COMPLETER : h.T @ d_out / 4
    db2 = np.mean(d_out, axis=0, keepdims=True)
```

 CC BY-NC-SA 4.0

```python

    d_h = d_out @ W2.T * sigmoid_d(h)
    dW1 = ______          # A COMPLETER : X.T @ d_h / 4
    db1 = np.mean(d_h, axis=0, keepdims=True)

    # Mise a jour
    W2 -= alpha * dW2; b2 -= alpha * db2
    W1 -= alpha * dW1; b1 -= alpha * db1

    if epoch % 1000 == 999:
        preds = (out > 0.5).astype(int).flatten()
        acc = np.mean(preds == y.flatten())
        print(f"Epoch {epoch+1} : "
              f"loss={loss:.4f}, acc={acc:.0%}")

print(f"\nPredictions : {out.flatten().round(3)}")
print(f"Attendu     : {y.flatten().astype(int)}")
```

Exercices Python autonomes — Niveau avancé

Exercice 11 — ★★★ **Avancé** — Réseau from scratch pour salaire_eleve [Python autonome]

Écrivez un programme Python complet qui :

a) Charge le dataset salaire (30 développeurs, 2 features : expérience et nb_langages) et la cible binaire (seuil 45 k€).
b) **Normalise** les features (soustraire la moyenne, diviser par l'écart-type).
c) Implémente un réseau **2-4-1** from scratch avec :
   — Forward pass (sigmoïde partout)
   — Backward pass (backpropagation)
   — Mise à jour par descente de gradient ($\alpha = 0{,}5$, 2000 époques)
d) Affiche la **loss et l'accuracy** toutes les 500 époques.
e) Trace **deux graphiques** côte à côte :
   — À gauche : courbe de loss par époque
   — À droite : nuage de points + régions de décision du réseau (utiliser `meshgrid` + `contourf`)
f) Comparez l'accuracy finale avec le **perceptron** du Jour 3 (∼87–90%).

⚠ **Consignes**

— Utilisez **uniquement** NumPy et Matplotlib.
— La normalisation est **essentielle** pour que le réseau converge.
— Initialisez les poids aléatoirement (`np.random.randn * 0.5`).

Exercice 12 — ★★★ **Avancé** — NumPy vs scikit-learn MLPClassifier [Python autonome]

Écrivez un programme qui compare deux implémentations :

a) **Méthode 1 — From scratch :** votre réseau 2-4-1 de l'exercice 11.

b) **Méthode 2 — scikit-learn :**

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
```

Utilisez `MLPClassifier(hidden_layer_sizes=(4,), max_iter=2000, activation='logistic', random_state=42)`.

c) Affichez un **tableau comparatif** :

Méthode	**Accuracy**	**Nb paramètres**
From scratch (2-4-1)	?	?
MLPClassifier (4,)	?	?
Perceptron (Jour 3)	?	?

d) Tracez les **surfaces de décision** des 3 modèles côte à côte (3 sous-figures avec `contourf`).

e) Répondez en commentaire : le réseau est-il meilleur que le perceptron ? La frontière est-elle linéaire ou courbe ? Combien de paramètres le réseau a-t-il vs le perceptron ?

Indice

Le perceptron a 3 paramètres (w_1, w_2, b). Le réseau 2-4-1 a $2 \times 4 + 4 + 4 \times 1 + 1 = 17$ paramètres. Plus de paramètres = plus de capacité à modéliser des frontières complexes, mais aussi plus de risque de surapprentissage.

Répartition : 4 exercices ★ fondamentaux 3 exercices ★★ intermédiaires 3 guidés Python ★★ 2 autonomes ★★★

© 2025 Mohamed EL AFRIT — IPSSI • www.mohamedelafrit.com • CC BY-NC-SA 4.0